

人工智能原理第一次项目作业实验报告-围棋 AI

杜雨桥 2023010966

2026.04

本项目完整代码托管于 GitHub，仓库地址为：<https://github.com/sparhalf/MyGo-AI>

1 第一小问-随机 Agent

1.1 设计思路

随机 Agent 先获取这一步的全部合法行动，然后从中随机选取。鉴于认输在“随机 Agent”里通常不具备参考价值，选择时若仍有其它合法走法，则 Agent 不主动选择认输。设计代码如下：

Listing 1: 随机 Agent 实现

```
def select_move(self, game_state: GameState) -> Move:
    """
    选择随机合法棋步
    """
    moves = game_state.legal_moves()
    non_resign_moves = [m for m in moves if not m.is_resign]
    if non_resign_moves:
        return random.choice(non_resign_moves)
    return random.choice(moves)
```

1.2 实验结果

使用 play.py 测试随机 Agent，结果如下：

```

=== 终局 ===
  1 2 3 4 5
1 X X X O O
2 . X X O O
3 X X O O O
4 O O O O O
5 . O O O O
胜者: white

===== 统计 =====
对局数: 1
黑方 (random) 胜: 0
白方 (random) 胜: 1
平局: 0
平均步数: 38.0
平均用时: 0.02s

```

图 1: 随机 Agent 对弈实验结果

可以看到对局正常进行到了终局，验证了规则的调用。

2 第二小问-MCST Agent

2.1 设计思路

本部分实现基于蒙特卡洛树搜索 (MCTS) 的围棋决策模块。每轮决策依次经历选择、扩展、模拟、反向传播四个步骤，在有限时间内通过随机采样逼近最优走法。

2.1.1 选择 (Selection)

选择阶段从根节点出发，沿树向下按照一定规则递归选取最佳子节点，直到到达一个没完全展开的叶节点或终局节点。我采用的是采用经典的 UCT 公式：

$$UCT(v) = \frac{w_v}{n_v} + c \cdot \sqrt{\frac{\ln N_p}{n_v}}$$

其中 w_v 为节点累积胜场数， n_v 为节点访问次数， N_p 为父节点访问次数， $c = \sqrt{2} \approx 1.414$ 为探索常数。对于从未被访问的节点 ($n_v = 0$)，直接赋予 $+\infty$ 的得分以保证优先探索，实现如下：

Listing 2: 选择最佳子节点流程

```

def best_child(self, c=1.414):
    if not self.children:
        return None

    parent_visits = max(1, self.visit_count)

```

```

best = None
best_score = -1e30
for child in self.children:
    if child.visit_count == 0:
        score = float("inf")
    else:
        score = child.value + c * math.sqrt(math.log(parent_visits) /
            child.visit_count)
    if score > best_score:
        best_score = score
        best = child
return best

```

2.1.2 扩展 (Expansion)

当选择阶段到达一个仍有未尝试走法的节点时，进入扩展阶段。我采用**逐步扩展**策略：每次仅从没尝试过的走法中取出一个，生成对应的子节点并加入子节点列表，而非一次性展开所有合法走法。这样可以避免在分支因子较大时一次性创建过多节点带来的内存开销。

Listing 3: 扩展流程

```

def expand(self):
    if not self.untried_moves:
        return None
    move = self.untried_moves.pop()
    next_state = self.game_state.apply_move(move)
    child = MCTSNode(next_state, parent=self)
    child.move = move
    # child 是“我方走了 move 之后”到达的局面
    child.player_just_played = self.game_state.next_player
    self.children.append(child)
    return child

```

值得注意的是，扩展时记录了 `player_just_played`，即“是哪一方走了这步棋到达了该节点”。这一字段后续在反向传播中用于正确分配胜负归属。

2.1.3 标准 MCST 模拟方法 (Simulation)

扩展得到新节点后，从该节点的局面出发进行随机模拟 (Rollout) 直至终局，以估计该局面的价值。标准版采用纯随机走子策略：每一步从当前合法走法中随机选取一步（排除认输），直到游戏结束，最终返回 `state.winner()` 作为本次模拟的结果。

Listing 4: 标准版纯随机 Rollout

```

state = game_state
while not state.is_over():
    moves = state.legal_moves()
    non_resign = [m for m in moves if not m.is_resign]
    state = state.apply_move(
        random.choice(non_resign) if non_resign else random.choice(moves)
    )
return state.winner()

```

模拟返回的是 `Player` 枚举类型（或 `None` 表示和棋），而非浮点数，这使得反向传播阶段可以直接与每个节点的 `player_just_played` 进行比较，无需额外的视角转换。

2.1.4 反向传播 (Backpropagation)

模拟结束后，将结果沿路径回传，更新沿途每个节点的统计数据。由于树中不同深度的节点轮到不同玩家走子，不同深度节点价值的归属必须随节点的归属方而变化，所以我在每个节点扩展时记录 `player_just_played`，反向传播时直接将模拟赢家与该字段比对，独立判断每个节点的胜负：

Listing 5: backup 的玩家视角比对

```

def backup(self, winner: Player | None):
    node = self
    while node is not None:
        node.visit_count += 1
        if node.player_just_played is None:
            pass # 根节点仅累计访问次数
        elif winner is None:
            node.value_sum += 0.5
        elif winner == node.player_just_played:
            node.value_sum += 1.0
        else:
            node.value_sum += 0.0
        node = node.parent

```

这样可以保证每个节点的 `value` ($= w_i/n_i$) 始终反映“走出这步棋的一方”的胜率，从而使得 `best_child` 中统一选取 UCT 最大的子节点在逻辑上完全自治——无论当前是黑方还是白方的回合，选择逻辑无需任何修改。

2.1.5 增强搜索效率的策略

标准版的纯随机 Rollout 在围棋中效果有限：随机走子产生的终局往往质量极低，对局面价值的估计噪声大，需要大量模拟次数才能收敛。因此我引入了两项增强策略：**启发式走子**与**深度截断**

结合快速局面评估，在不显著增加单次模拟耗时的前提下大幅提升了价值估计的准确性。

策略一：启发式走子 在模拟阶段，将纯随机选子替换为带有围棋领域知识的启发式策略。

核心思路是**优先选择能够提取对方棋子的走法，若不能提子，优先选“落子后气更多/避免自紧”的走法**：对每一个候选落子点，先记录对方当前的棋子数量，执行该走法后再次统计，若对方棋子减少则认为该走法可以提子，将其加入优先候选集合。若存在可提子的走法，则从中随机选取一步；若没有则退而求其次，寻找落子后气更多/避免自紧的走法。如果上边都做不到就退化为从所有合法落子点中随机选取；只有在无落子点可选时才考虑 pass。

Listing 6: 启发式走子策略

```
before = _count_stones(game_state, opponent)
capture_moves = []
for m in moves:
    if not m.is_play:
        continue
    next_state = game_state.apply_move(m)
    after = _count_stones(next_state, opponent)
    if after < before:
        capture_moves.append(m)
if capture_moves:
    return random.choice(capture_moves)

play_moves = [m for m in moves if m.is_play]
if play_moves:
    scored = []
    for m in play_moves:
        ns = game_state.apply_move(m)
        s = ns.board.get_go_string(m.point)
        libs = s.num_liberties if s is not None else 0
        # 先避免自紧（气<=1），再偏好气更多
        scored.append(((libs <= 1, -libs), m))

    scored.sort(key=lambda item: item[0])
    best_key = scored[0][0]
    best_moves = [m for k, m in scored if k == best_key]
    return random.choice(best_moves)
return random.choice(moves)
```

策略二：深度截断与快速局面评估 围棋终局往往需要数百步，纯随机 Rollout 走至终局的时间开销极大，限制了单位时间内可完成的模拟轮数。我对模拟深度进行截断：最多走 rollout_depth

(默认 30) 步后强制停止，再用快速局面评估函数估计当前局面的胜负。

Listing 7: 深度截断与子数差评估

```
state = game_state
for _ in range(self.rollout_depth):
    if state.is_over():
        break
    move = self._select_rollout_move(state)
    state = state.apply_move(move)

if state.is_over():
    return state.winner()

# 深度截断后用子数差作为快速评估
black = _count_stones(state, Player.black)
white = _count_stones(state, Player.white)
score = black - white
if score > 0:
    return Player.black
if score < 0:
    return Player.white
return None
```

截断后的局面评估采用**棋子数差**作为代理指标：黑方棋子多则认为黑方领先，反之则白方领先，子数相等则视为和棋。在 5×5 的小棋盘上，棋子数差与真实的终局胜负具有较强的相关性，是一种低计算开销的合理近似。

2.2 实验结果

2.2.1 不同算法 Agent 对比

我在 5×5 的棋盘上分别对三种组合的对战模式分别模拟了 100 次棋局，模拟过程中不断交换黑白方起手，MCST Agent 每轮模拟 200 次，每步上限 10 秒，增强 MCST Agent 在 15 层截断，得到的胜负结果如下：

表 1: 不同对战组合结果

	胜率	每盘平均用时	平均步数	平均每步用时
随机 vs 标准 MCST	1% vs 99%	10.55s	26.1	0.40s
随机 vs 增强 MCST	0 vs 99%(1 平局)	15.99s	29.2	0.54s
标准 MCST vs 增强 MCST	14% vs 78%(8 平局)	30.20s	34.8	0.87s

可见，标准 MCTS 对随机智能体取得了 99% 的胜率，证明即使是不加任何优化的蒙特卡洛树搜索，在足够的模拟次数下也能形成对随机走子的压倒性优势，验证了算法实现的基本正确性。

其次，增强 MCTS 对随机智能体同样取得了 99% 的胜率（1 局平局），与标准版胜率持平，但每盘平均用时从 10.55s 上升至 15.99s，平均步数也从 26.1 增至 29.2 步。这说明启发式走子策略使模拟过程产生了质量更高、更接近真实对局的棋局，对局因此更趋完整而非因随机走子快速崩溃，单步耗时的增加（0.40s $\rightarrow$ 0.54s）也反映了启发式评估带来的额外计算开销。

最关键的对比在于标准 MCTS 与增强 MCTS 的直接对局：增强版以 78% 对 14%（8 局平局）的胜率明显胜出。这一结果表明，启发式走子与深度截断结合快速局面评估的组合策略，能够在相同模拟次数预算下产生更准确的价值估计，搜索质量的提升足以抵消单步耗时增加的代价。平均步数增至 34.8 步、每盘用时达 30.20s，也说明两个较强智能体之间的对局博弈更为充分，不会因一方快速失误而提前结束。

2.2.2 增强 MCST Agent 截断层数对比

为了探究截断层数的影响，我固定截断层数为 10 层的增强 MCST Agent 与 5、20、30、40 层的增强 MCST Agent 进行对弈，其他参数同实验 2.2.1。结果如下：

表 2: 不同对战组合结果

	胜率	每盘平均用时	平均步数	平均每步用时
10 层 vs 5 层	50% vs 45%(5 平局)	23.98s	35.2	0.68s
10 层 vs 20 层	46% vs 46%(8 平局)	35.47s	35.3	1.00s
10 层 vs 30 层	43% vs 52%(5 平局)	44.98s	34.7	1.30s
10 层 vs 40 层	40% vs 57%(3 平局)	54.31s	33.2	1.63s

由表可知，随着截断层数的增加，增强 MCTS Agent 的胜率呈现出单调上升的趋势：相较于固定的 10 层基准，5 层的胜率仅为 45%，与 10 层基本持平；而 20 层、30 层、40 层的胜率依次提升至 46%、52% 与 57%，10 层基准的胜率则从 46% 逐步下降至 40%，趋势较为明显。

这一结果说明，在模拟次数相同的前提下，更深的 Rollout 截断层数能够让每次模拟覆盖更接近终局的局面，从而获得质量更高的回报信号，进而改善节点价值估计的准确性。然而，代价是每步平均用时随截断层数线性增长——从 5 层的 0.68s 逐步上升至 40 层的 1.63s，每盘用时也从约 24s 增至 54s。因此，截断层数的选取本质上是**搜索质量与计算开销之间的权衡**：层数过浅时模拟终止过早，局面评估噪声较大；层数过深时单次模拟耗时增加，相同时间预算内能完成的模拟轮次减少，反而可能削弱搜索树的广度。综合来看，在本实验的时间限制与棋盘规模下，15 ~ 20 层是较为合理的平衡点。

3 第三小问-MINIMAX Agent

3.1 设计思路

我实现了一个基于 Minimax 搜索与 Alpha-Beta 剪枝的围棋智能体，并设计了一套结合气数与子数的启发式评估函数，以及基于提子与气数的走法排序策略，以尽可能提升 Alpha-Beta 的剪枝效率。

3.1.1 基础 Minimax 算法

基础 Minimax 算法采用递归方式构建博弈树。搜索时以 `maximizing_player` 标志位区分当前层是最大化层(我方)还是最小化层(对方),当局面已结束或搜索深度耗尽时,调用 `_terminal_or_eval` 返回叶节点的估值。

Listing 8: 基础 Minimax 递归结构

```
if maximizing_player:
    value = -math.inf
    for move in moves:
        value = max(
            value,
            self.minimax(
                game_state.apply_move(move), depth - 1, maximizing_player=
                    False
            ),
        )
    return value
else:
    value = math.inf
    for move in moves:
        value = min(
            value,
            self.minimax(
                game_state.apply_move(move), depth - 1, maximizing_player=True
            ),
        )
    return value
```

终局与截断深度的处理统一由 `_terminal_or_eval` 完成：若局面已结束，则对我方胜利返回 $+10^6$ ，对我方失败返回 -10^6 ，和棋返回 0；若未终局但深度耗尽，则调用评估函数给出估值。这种设计将终局判断与启发式评估解耦，便于替换评估函数。

Listing 9: 终局与截断评估

```
def _terminal_or_eval(self, game_state: GameState) -> float:
    if game_state.is_over():
        winner = game_state.winner()
        if winner is None:
            return 0.0
        return 1e6 if winner == self._root_player else -1e6
    return float(self.evaluator(game_state))
```

3.1.2 Alpha-Beta 剪枝

Alpha-Beta 算法维护两个界： α （最大化层已确保的下界）和 β （最小化层已确保的上界），当 $\alpha \geq \beta$ 时，当前分支不可能影响上层决策，可直接剪除：

Listing 10: Alpha-Beta 剪枝条件

```
if maximizing_player:
    ...
    alpha = max(alpha, value)
    if alpha >= beta:
        break    # beta 剪枝
else:
    ...
    beta = min(beta, value)
    if beta <= alpha:
        break    # alpha 剪枝
```

在此基础上，我还结合了**置换表 (Transposition Table)** 进一步消除冗余计算。使用 Zobrist 哈希对局面进行索引，在进入递归前先查询缓存，若命中则直接返回缓存值；搜索完成后将结果写入缓存，并遵循“深度优先”原则：只有新结果的搜索深度不低于缓存中的深度时，才更新缓存条目，以保证缓存中始终保存最精确的估值。

Listing 11: 置换表的查询与写入

```
zobrist_hash = game_state.board.zobrist_hash()
cached = self._cache.get(
    (zobrist_hash, depth, maximizing_player, self._root_player)
)
if cached is not None:
    return cached[1]

# ... 搜索完成后 ...
self._cache.put(
    (zobrist_hash, depth, maximizing_player, self._root_player), depth, value
```

)

缓存键包含了 `depth`、`maximizing_player` 和 `_root_player` 四个维度，确保不同搜索上下文下的估值不会互相污染。

3.1.3 启发式局面评估函数

当搜索深度耗尽而局面尚未结束时，需要一个评估函数估计当前局面的优劣。默认评估函数综合考虑子数差与气数差两个维度：

$$\text{eval}(s) = (S_{\text{me}} - S_{\text{opp}}) + 0.2 \times (L_{\text{me}} - L_{\text{opp}})$$

其中 S 为棋子数， L 为气数之和，0.2 为气数的权重系数。

Listing 12: 启发式评估函数核心逻辑

```
seen_strings = set()
for r in range(1, board.num_rows + 1):
    for c in range(1, board.num_cols + 1):
        pt = Point(r, c)
        stone = board.get(pt)
        if stone is None:
            continue
        if stone == me:
            me_stones += 1
        else:
            opp_stones += 1

        s = board.get_go_string(pt)
        key = (s.color, s.stones)
        if key in seen_strings:
            continue
        seen_strings.add(key)
        if s.color == me:
            me_libs += s.num_liberties
        else:
            opp_libs += s.num_liberties

return (me_stones - opp_stones) + 0.2 * (me_libs - opp_libs)
```

子数差反映了直接的物质优势，是围棋中最直接的胜负指标；气数差则反映了棋形的活力与被提吃的风险——气少的棋串更容易被围杀。气数的权重系数取 0.2 而非更大的值，是为了防止智能

体过度追求”虚气”（如眼位）而忽略实际的子数优势。此外，统计气数时使用 `seen_strings` 集合对棋串去重，避免对同一棋串的气重复累计。

3.1.4 走法排序

Alpha-Beta 剪枝的效率高度依赖走法的搜索顺序：若最优走法总是最先被检查，剪枝效果可从 $O(b^d)$ 降至接近 $O(b^{d/2})$ 。我在 `_get_ordered_moves` 中设计了一套三级优先级排序策略：

1. **提子数**（越多越优先）：能提取对方棋子的走法优先度最高，对应最直接的物质收益；
2. **自紧惩罚**（避免气 ≤ 1 的走法）：落子后己方棋串仅剩一口气，极易被反提，给予降级处理；
3. **落子后气数**（气越多越优先）：在无提子机会时，优先选择落子后气数更充裕的位置，有助于保持棋形弹性。

Listing 13: 走法排序的评分 key 构造

```
captured = max(0, before_opp - after_opp)
s = ns.board.get_go_string(m.point)
libs = s.num_liberties if s is not None else 0
# key 越小越优先
key = (0, -captured, libs <= 1, -libs)
scored.append((key, m))
```

排序 key 为四元组，Python 的元组比较天然支持多级排序：首位 0 使所有落子优先于 pass（pass 的首位为 10）；第二位 `-captured` 使提子数多的走法排在前面；第三位 `libs <= 1` 将自紧走法下压；第四位 `-libs` 在气数相近时进一步细化顺序。

这一排序策略在实践中能显著提升 Alpha-Beta 的剪枝率：提子走法往往对应搜索树中估值最高的分支，优先搜索这类走法可使 α 界尽快抬升，从而剪除更多后续分支，在有限的深度预算内实现更深的有效搜索。

3.2 实验结果

我在 5x5 的棋盘上分别对三种组合的对战模式分别模拟了 100 次棋局，模拟过程中不断交换黑白方起手，MINIMAX Agent 的 max depth 取 3，MCST Agent 参数与实验 2.2 相同，得到的胜负结果如下：

表 3: 不同对战组合结果

	胜率	每盘平均用时	平均步数	每步平均时间
MINIMAX vs 随机	92% vs 6%(2 平局)	0.71s	34.7	0.02s
MINIMAX vs 标准 MCST	63% vs 34%(1 平局)	13.06s	34.7	0.38s
MINIMAX vs 增强 MCST	59% vs 38%(3 平局)	18.34s	34.5	0.53s

由表 2 可以得出以下结论：

首先，Minimax 智能体对随机智能体取得了 92% 的胜率，仅有 6% 的失负与 2 局平局，证明基于精确树搜索的 Minimax 算法在对抗无策略的随机走子时具有压倒性优势。值得注意的是，Minimax 的每步平均用时仅为 0.02s，远低于 MCTS 的 0.40s 以上，体现了在小棋盘、有限分支因子下确定性搜索的高效性。

其次，Minimax 对标准 MCTS 取得了 63% vs 34% 的胜率优势，对增强 MCTS 同样保持 59% vs 38% 的领先。这一结果在本实验的参数设置下是合理的：MCTS 每轮仅限 200 次模拟，统计样本相对有限，价值估计存在一定噪声；而 Minimax 在 5×5 棋盘上的分支因子较小，深度为 3 的精确搜索配合气数与子数的启发式评估函数，已能对近期局面做出较为准确的判断，因此在当前参数下展现出更强的棋力。

对比两组 MCTS 的结果，增强 MCTS 相较标准 MCTS 将对 Minimax 的胜率从 34% 小幅提升至 38%，负率从 63% 降至 59%，说明启发式 Rollout 策略确实改善了 MCTS 的局面估计质量，但在模拟次数受限的条件下提升幅度有限。若将 MCTS 的模拟次数大幅提高，其胜率预计将显著反超 Minimax，因为 MCTS 的搜索质量随模拟次数的增加而单调收敛至最优，而固定深度的 Minimax 则受限于评估函数的精度上限。

为了进一步探究是否是棋盘太小导致 MCST Agent 胜率较低，我将棋盘扩充为 10×10 进行了 10 局模拟，结果如下：

表 4: 10×10 棋盘对战组合结果

	胜率	每盘平均用时	平均步数	每步平均时间
MINIMAX vs 标准 MCST	50% vs 50%	753.22s	155.9	4.83s

可以看到此时标准 MCST Agent 的胜率来到了 50%，但一盘的用时也大幅提高到了 753.22s，平均步数达到 155.9 步，每步平均耗时 4.83s。

这一结果表明，棋盘规模对 MCTS 的搜索质量有显著影响。在 5×5 的小棋盘上，合法走法少、终局来得快，Minimax 的深度优先搜索能够在有限深度内覆盖更大比例的关键局面，因此具备明显优势；而棋盘扩展至 10×10 后，合法走法数量大幅增加，Minimax 的固定深度搜索难以触及足够深的局面，评估精度下降，MCTS 的大量随机模拟反而能更好地弥补这一不足，双方因此趋于均势。然而， 10×10 棋盘下每盘超过 750 秒的耗时也暴露了两种算法在大棋盘上的计算瓶颈——

Minimax 受限于指数级的搜索空间，MCTS 则面临更长的 Rollout 路径，在实际应用中均需进一步优化才能满足时效要求。

4 图形化界面

4.1 功能描述

我基于 Python 标准库 `tkinter` 实现了一套围棋对弈图形化界面 (`gui.py`)。

界面支持以下功能：

- **双模式对弈**：支持**人机对弈**与**AI 对弈**两种模式。人机对弈中玩家执黑先行，通过鼠标点击落子，AI 自动应手；AI 对弈模式下双方均由 Agent 控制，用户每次点击”下一步”按钮驱动 AI 走一步，便于逐步观察搜索结果。
- **多种 AI 类型**：提供四种 Agent 供选择——`random`（随机智能体）、`mcts_std`（标准 MCTS）、`mcts_enh`（增强 MCTS）与 `minimax`（Minimax + Alpha-Beta 剪枝）。
- **对局信息实时显示**：界面顶部实时显示当前回合（黑/白方及人类/AI 标注）、黑白双方提子数、已下手数以及本局累计用时。
- **对局控制**：人机对弈模式下支持**悔棋**（最多回退两步，自动回到人类回合）、**Pass**（停一手）、**认输**与**新游戏**；AI 对弈模式下悔棋、Pass 与认输功能禁用，仅保留”下一步”与”新游戏”。
- **异常提示**：若玩家点击非法落子位置，弹出警告提示；若当前局面无合法落子点，系统自动提示玩家选择 Pass，且同一局面只提示一次，避免重复弹窗干扰。

示例如下：



图 2: 人机对弈菜单

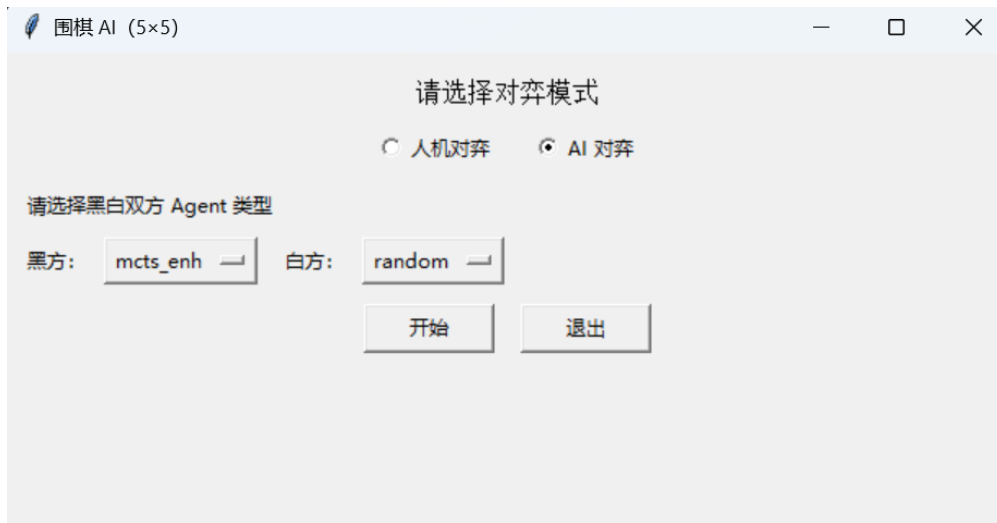


图 3: AI 对弈菜单

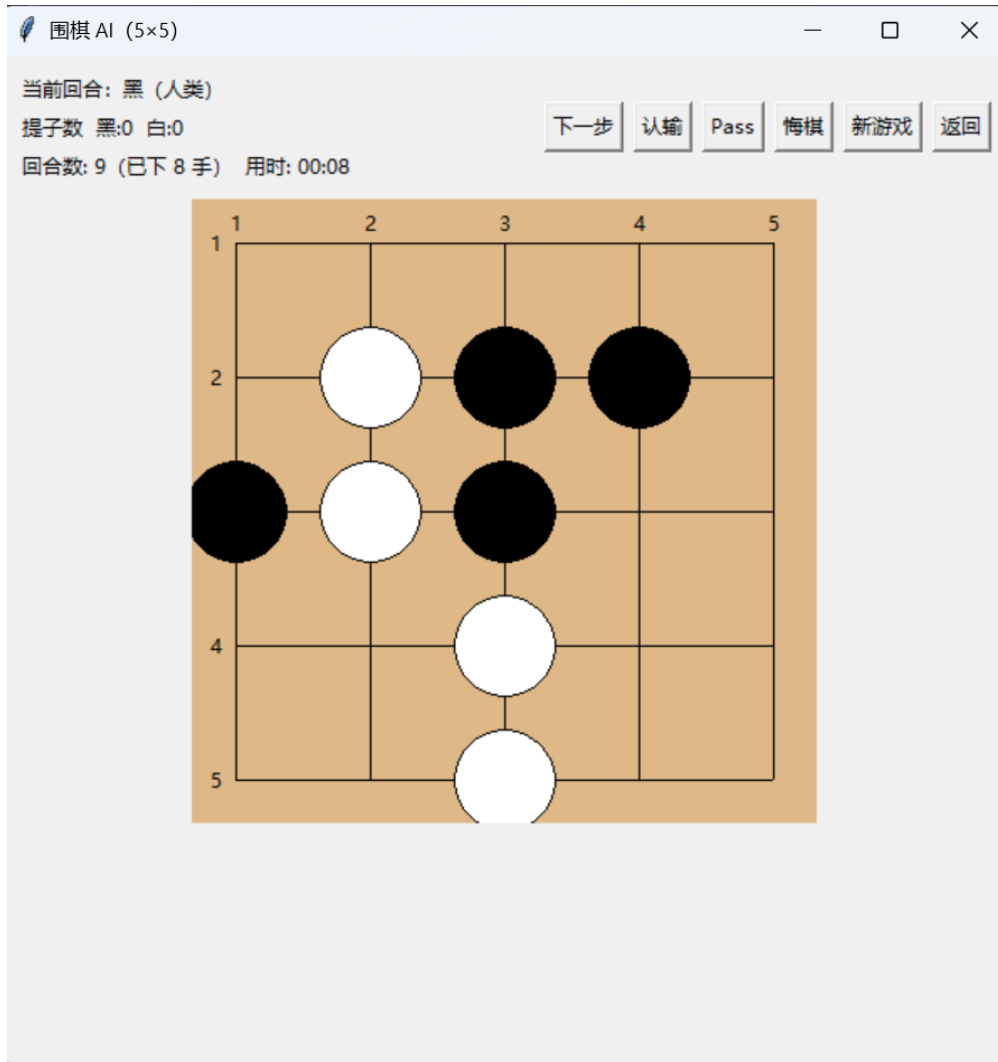


图 4: 下棋实例

4.2 使用说明

在项目根目录下通过命令行启动图形化界面：

Listing 14: 启动 GUI

```
# 默认 5x5 棋盘
python gui.py

# 指定棋盘大小 (如 10x10)
python gui.py --size 10
```

启动后进入**模式选择页**，操作流程如下：

1. **选择对弈模式**：在”人机对弈”与”AI 对弈”之间二选一。

- 选择**人机对弈**后，从下拉菜单中选择对手 AI 类型 (`random` / `mcts_std` / `mcts_enh` / `minimax`)。
 - 选择 **AI 对弈**后，分别为黑方和白方各选择一种 Agent 类型。
2. 点击”**开始**”进入棋盘界面。
 3. **人机对弈操作**:
 - 鼠标左键点击棋盘交叉点进行落子；点击非法位置将弹出提示。
 - 点击”**Pass**”可停一手；点击”**认输**”结束对局。
 - 点击”**悔棋**”可回退至上一个人类回合（最多回退两步）。
 4. **AI 对弈操作**:
 - 每次点击”**下一步**”，当前回合的 AI 走一步；Pass、悔棋与认输按钮在此模式下禁用。
 5. 任意模式下均可点击”**新游戏**”按当前配置重新开始，或点击”**返回**”回到模式选择页重新配置。
 6. 对局结束后弹出终局对话框，显示胜者（或平局）信息。

5 AI 使用说明

5.1 使用的 AI 工具

本次实验主要使用了以下两款 AI 工具：

- **Cursor Agent**：一款集成了大语言模型的智能代码编辑器，支持在编辑器内直接与 AI 对话并对代码进行局部修改。
- **Gemini 3.1 Pro**：Google 推出的多模态大语言模型，用于查询算法相关的概念与设计细节。

5.2 AI 辅助的具体使用场景

5.2.1 图形化界面开发

围棋的图形化对战界面（棋盘渲染、落子交互、胜负提示等）涉及较多与核心算法无关的 UI 细节代码。为节省时间，我向 Cursor Agent 描述了界面的布局需求，由其生成了界面的基础框架，之后再根据实际运行效果手动调整了布局参数与交互逻辑。

5.2.2 代码注释完善

在核心算法模块完成后，我使用 Cursor Agent 为关键函数（如 `backup`、`alphabeta` 等）补充了文档字符串（docstring）与行内注释，以提升代码的可读性。注释内容经过人工审阅，确保与实际逻辑一致。

5.2.3 算法正确性验证

在实现 MCTS 的反向传播逻辑时，我对”不同层节点的胜负视角是否正确”存在疑问，将相关代码片段提交给 Cursor Agent 进行审查，AI 指出了初版中 `player_just_played` 赋值时机的一处潜在问题，经核对后对逻辑进行了修正。

5.2.4 算法设计细节咨询

在设计 MCTS 与 Minimax 智能体之前，我向 Gemini 3.1 Pro 咨询了以下问题：

- UCT 公式中探索常数 C 的取值对搜索行为的影响；
- Alpha-Beta 剪枝中置换表（Transposition Table）的存储策略；
- Rollout 深度截断后局面评估的常用方法。

上述咨询以理解概念为目的，具体的代码实现均由本人根据理解独立完成。

5.3 AI 使用感受

总体而言，AI 工具在我的项目中扮演了”助手”而非”主导者”的角色。在图形界面这类与算法关系不大的工程性任务上，AI 的辅助确实节省了大量时间；而在核心算法的设计与调试上，AI 更多起到了”答疑”和”代码审查”的作用，真正理解算法、决定设计方案的过程仍需要自己独立思考。

值得一提的是，AI 给出的建议并非总是可以直接采用——例如 Gemini 在解释置换表时给出的示例代码与本项目的接口不完全兼容，仍需自行改写。这让我意识到，AI 工具的价值在于”加速已知方向上的执行”，而对问题的理解与判断依然不可替代。